ASSIGNMENT 5 - PART B

B3 - COMPARE BARE METAL, VIRTUAL MACHINE AND CONTAINER

Implementation and Experimental Investigation | 8 Marks | K5 - Evaluate

1. Objective and Task Requirements

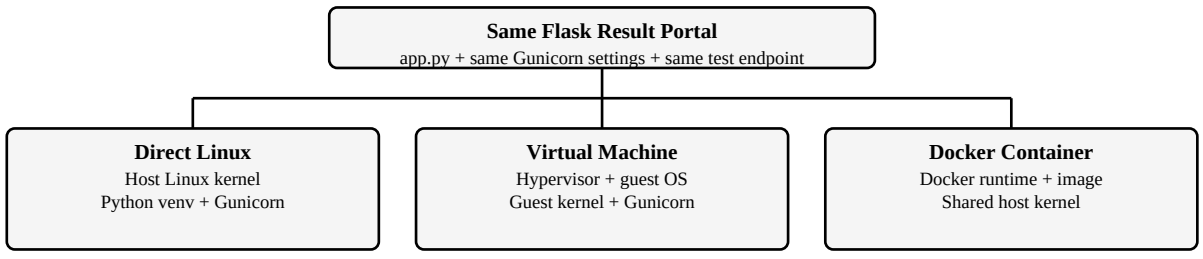
B3 evaluates three ways of deploying the same web application: directly on Linux, inside a virtual machine (VirtualBox or KVM), and inside a Docker container. It requires measurement of start-up time, memory use, image or disk size and requests per second; deployment into a local Kubernetes cluster with three replicas; deletion and automatic recreation of a Pod; scaling to six replicas under load; and an engineering trade-off explaining when each method should be used.

Section	Marks	Evidence required
B3.1 - Deploy Three Ways	2	Same web application running directly on Linux, in a VM and in a Docker container.
B3.2 - Measure	2	Start-up time, memory use, image/disk size and requests per second.
B3.3 - Scale with Kubernetes	2	Three Pods, Pod deletion/recreation and scaling to six under load.
B3.4 - Engineering Trade-Off	2	Interpret measurements and identify suitable use cases.

The supplied case study describes a result portal serving 150,000 students in a two-hour window and currently failing on one physical Linux server. B3 is a comparative deployment experiment, so the same application and workload must be used in each environment. A successful deployment alone is insufficient; measured evidence and interpretation are required.

2. Common Application and Controlled Test Conditions

A single Flask application is provided as `app.py`. It exposes a home page (`/`), a health endpoint (`/health`) and a deterministic CPU-work endpoint (`/work?n=10000`). The Docker image, VM and direct Linux deployment all use this same source and two Gunicorn workers. The CPU-work endpoint helps produce a repeatable request workload rather than benchmarking only an almost-empty health response.



Measure startup, memory, disk/image footprint and requests/second

Keep code, worker count, endpoint, request count and concurrency identical

Kubernetes: same container image, Deployment with 3 replicas; then 6 under load

For a fair comparison, keep the following constant: the application source and dependency versions, Gunicorn worker count, request path and query parameter, request count, concurrency, benchmark client, machine background load and warm-up procedure. Repeat each throughput test at least three times and report the median. State the VM's vCPU/RAM allocation and whether container image layers were cached.

The benchmark should be run from outside the deployment being measured. ApacheBench (`ab`) is used in the supplied helper script; install it with `apache2-utils`. Use the same request count and concurrency, for example 2,000 requests at concurrency 20, against each environment's URL.

3. B3.1 - Deploy the Same Application Three Ways

3.1 Directly on Linux

On a native Linux host, create a Python virtual environment, install the shared requirements and start Gunicorn. This deployment has no separate guest operating system and no Docker container layer. It still uses normal Linux process isolation, permissions and host scheduling; it should not be described as having no operating-system overhead.

```
python3 -m venv .venv
. .venv/bin/activate
pip install -r requirements.txt
gunicorn --workers 2 --bind 0.0.0.0:8000 app:app

curl http://127.0.0.1:8000/health
```

3.2 Inside a virtual machine

Create an Ubuntu VM in VirtualBox or KVM and record its configured vCPU count, RAM and virtual disk size. Copy the same application files into the guest, install the same Python dependencies and run the same Gunicorn command on guest port 8000. With VirtualBox NAT, forward host port 8001 to guest port 8000; a bridged network address can also be used. The included systemd unit makes the application ready automatically when the guest OS boots, allowing VM start-up measurement to include guest boot and application readiness.

VM memory must be reported using two related but distinct measurements: the configured guest RAM and the RAM used inside the guest (`free -h`). Host-side VirtualBox/QEMU process RSS can also be recorded. The virtual disk's provisioned capacity is not the same as the actual VDI/QCOW2 file size; record both where possible.

3.3 Inside a Docker container

Build the common app image and run it with host port 8002 mapped to container port 8000. CPU and memory limits are set for the experiment so the resource measurement has explicit constraints.

```
docker build -t simats/result-portal:1.0 .
docker run -d --name result-portal-container \
  -p 8002:8000 --cpus=0.50 --memory=512m \
  simats/result-portal:1.0

curl http://127.0.0.1:8002/health
docker stats --no-stream result-portal-container
docker image ls simats/result-portal:1.0
```

The container uses the same app image and Gunicorn settings. It shares the host Linux kernel, while the image supplies the user-space filesystem and application dependencies. Record both image size and actual container memory from `docker stats`; the values are different measures.

4. B3.2 - Measurement Method

4.1 Start-up time

Define start-up time consistently as the elapsed time from issuing the start command from a stopped state until the `/health` endpoint first returns HTTP 200. For direct Linux and Docker, the supplied `measure_startup.py` starts the corresponding process/command and polls the health URL. For the VM, power off the guest first, start it with `VBoxManage startvm`, and poll the forwarded health URL; this includes the guest OS boot and systemd application start. Repeat at least three times and report the median.

4.2 Memory usage

Method	Memory evidence
Direct Linux	Gunicorn process RSS using `ps -o rss= -p PID`, and host `free -h`.
VM	Configured guest RAM, guest memory usage from `free -h`, and host VirtualBox/QEMU process RSS.
Docker	`docker stats --no-stream` for the container; optionally inspect `docker inspect --size`.
Kubernetes	`kubectl top pods` and `kubectl top nodes` if the Metrics API is installed.

Do not compare unlike quantities as if they were identical. For example, configured VM RAM is an allocation, guest used memory is an in-guest observation, and container RSS/working-set values are runtime measures. State the metric used in every table cell.

4.3 Disk or image size

For direct Linux, record the application and virtual-environment footprint using `du -sh app.py .venv`. For a VM, record the configured virtual disk capacity and actual host file size (VDI or QCOW2). For Docker, record the image size from `docker image ls simats/result-portal:1.0`. Kubernetes Pods refer to the same container image; the image is not copied as a separate unique image per Pod on a node.

4.4 Requests per second

```
sudo apt install -y apache2-utils
scripts/benchmark_requests.sh baremetal http://127.0.0.1:8000/work?n=10000 2000 20
scripts/benchmark_requests.sh vm        http://127.0.0.1:8001/work?n=10000 2000 20
scripts/benchmark_requests.sh docker    http://127.0.0.1:8002/work?n=10000 2000 20
```

ApacheBench reports Requests per second and latency percentiles. Use the same request count (2,000), concurrency (20), keep-alive option, endpoint and test client. Warm up the application with a few requests first. Do not run the three benchmarks simultaneously on a constrained laptop, because the benchmark processes would compete with each deployment.

5. B3.3 - Kubernetes with Three Replicas, Recovery and Scaling

The completed `kubernetes/deployment.yaml` includes the Deployment and LoadBalancer Service requested in the paper. TODO 1 sets `replicas: 3` and `selector.matchLabels.app: result-portal`. TODO 2 sets CPU/memory requests to 250m and 256Mi, and limits to 500m and 512Mi. The readiness probe checks `/health` on port 8000.

```
spec:
  replicas: 3
  selector:
    matchLabels:
      app: result-portal
  template:
    metadata:
      labels:
        app: result-portal
```

```
  spec:
    containers:
      - name: portal
        image: simats/result-portal:1.0
        ports:
          - containerPort: 8000
        resources:
          requests:
            cpu: 250m
            memory: 256Mi
          limits:
            cpu: 500m
            memory: 512Mi
```

Build the image with the exact tag from the YAML. Load it into `kind` or `minikube` so the local cluster can run the image without pulling an unrelated registry copy. Apply the file and wait until three Pods are Ready. Then delete one Pod; the Deployment/ReplicaSet controller should create a replacement to restore the desired count.

```
docker build -t simats/result-portal:1.0 .
kind create cluster --name b3
kind load docker-image simats/result-portal:1.0 --name b3
kubectl apply -f kubernetes/deployment.yaml
kubectl rollout status deployment/result-portal --timeout=180s
kubectl get pods -l app=result-portal -o wide

POD=$(kubectl get pods -l app=result-portal -o jsonpath='{.items[0].metadata.name}')
kubectl delete pod "$POD"
kubectl get pods -l app=result-portal -w
```

For a minikube cluster, use `minikube start` and `minikube image load simats/result-portal:1.0` instead of the kind commands. The manifest retains `type: LoadBalancer` as the question specifies. Port-forward the Service locally for the benchmark, which works with both local cluster options.

```
kubectl port-forward service/result-portal 8003:80

# In another terminal, start a sustained test:
ab -n 50000 -c 50 -k http://127.0.0.1:8003/work?n=10000

# While the test is running:
kubectl scale deployment/result-portal --replicas=6
kubectl get pods -l app=result-portal -w
kubectl get deployment result-portal
kubectl describe pod POD_NAME
```

The six-replica observation must be captured while the request test is active. The Pod description should show the configured requests and limits. `kubectl top pods` is optional if the local cluster has a working Metrics API; if metrics are unavailable, record that limitation instead of inventing values.

A local single-node kind/minikube cluster demonstrates Kubernetes desired-state reconciliation and Pod replacement. It does not establish multi-node high availability. To test recovery from a worker-node failure, use a multi-node cluster and actually stop or remove a worker node.

6. B3.4 - Engineering Trade-Off and Alternative

Deployment	Advantages	Limitations / suitable use
Bare metal Linux	No guest OS or container runtime layer; direct use of host resources; simple process startup and debugging.	Applications share the host OS and need dependency/resource management. Use for workloads needing direct hardware access or predictable host performance.
Virtual machine	Separate guest kernel and OS environment; can run different distributions and provides a clearer isolation boundary.	Guest OS boot, memory and virtual disk overhead. Use for stronger workload isolation, legacy systems or different OS requirements.
Docker container	Fast start, reusable image, compact per-instance footprint and repeatable packaging.	Shares host kernel; runtime/daemon and image security matter. Use for portable services, CI/CD and rapid scaling.
Kubernetes	Maintains desired replicas, replaces failed Pods and schedules replicas across available nodes.	Requires cluster operations and spare capacity. Scale-out is limited by worker nodes, database capacity and networking.

6.1 Docker Compose scaling alternative

The alternative in the question is `docker compose up --scale portal=6` on one server. Compose can run six copies of the portal service on that host, but the maximum load is bounded by that machine's total CPU, memory, disk and network resources. It does not provide multi-node scheduling by itself. A load balancer or reverse proxy must distribute requests among replicas, and the service must not bind every replica to the same fixed host port.

If a container fails, a restart policy can restart it on the same server. If the single server fails, all six replicas are lost together. Kubernetes on a multi-node cluster can schedule replacement Pods on healthy nodes if resources and networking are available; a single-node local cluster only demonstrates Pod recreation on the same node. Recovery time should be measured from failure detection until the replacement Pod/container is Ready and the health endpoint responds.

6.2 Complexity and capacity reasoning

For N identical application instances, the controller or orchestrator maintains and starts instances with total instance-management work that grows approximately O(N). The application request algorithm is unchanged by deployment mode; the measured throughput depends on per-request work, scheduling overhead, network and storage paths, worker count and available hardware. Increasing replicas from three to six can increase aggregate throughput only while the host/cluster and database have unused capacity. If CPU, memory, network or database is already saturated, additional replicas may provide little gain or may worsen contention.

Kubernetes uses a desired replica count and continuously reconciles actual Pods toward it. Docker Compose scaling on one host also increases replica count, but it cannot move replicas to another machine without an external orchestration layer. The practical maximum load must therefore be taken from the measured requests/second and latency at a stated concurrency, not guessed from the replica count alone.

7. Experimental Results Table - Complete After Running

Metric	Bare metal Linux	Virtual machine	Docker container
Start-up time (median of 3)	_____ s	_____ s	_____ s
Memory metric and value	RSS: _____ MB	Guest used: ____ MB; host RSS: ____ MB	Stats: ____ MB
Disk / image size	App + venv: ____ MB	VDI/QCOW2 actual: ____ MB; max: ____ GB	Image: ____ MB
Requests per second (median)	_____	_____	_____
95th percentile latency	_____ ms	_____ ms	_____ ms

Kubernetes observation	Result to record
Initial replicas	3 Pods Ready: Yes / No; capture `kubectl get pods`.
Pod recovery	Deleted Pod: _____; replacement Pod: _____; time to 3 Ready: _____ s.
Scale under load	Request test: _____ requests, concurrency _____; scale to 6; time to 6 Ready: _____ s.
Resources	kubectl describe pod shows CPU request 250m / limit 500m and memory request 256Mi / limit 512Mi.

Do not enter sample or estimated values in these fields. Capture outputs on the machine used for the assignment, repeat tests and report actual median results. The final explanation should connect the observed start-up, memory, size and throughput numbers to the deployment trade-offs.

8. Important Environment Qualification

A deployment run directly inside a native Linux installation can be reported as bare metal. WSL2 runs inside a lightweight virtual machine; a Linux process launched directly in WSL2 is therefore not a true bare-metal deployment. Similarly, Docker Desktop on Windows commonly uses a Linux VM for Linux containers. If those are the available environments, label the results accurately (for example, “direct WSL2 Linux”, “VirtualBox Ubuntu VM”, and “Docker container via WSL2”) rather than claiming a physical bare-metal comparison.

9. Conclusion

B3 compares the same Flask portal on direct Linux, in a VM and in Docker, then evaluates Kubernetes orchestration. Bare metal avoids the guest-OS/container layers; VMs add a guest kernel and isolation boundary; containers share the host kernel and enable fast, repeatable deployment. Kubernetes maintains the desired replica count and can replace failed Pods, while six replicas under load demonstrate horizontal scaling only within available cluster capacity. The correct engineering choice depends on actual measurements, security requirements, resource limits and recovery objectives.

10. Technical References and Task Alignment

[1] Kubernetes Documentation, Deployments - desired state and ReplicaSet reconciliation.
https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
[2] Kubernetes Documentation, Services - stable networking and LoadBalancer Service type.
https://kubernetes.io/docs/concepts/services-networking/service/
[3] Kubernetes Documentation, Resource Management for Pods and Containers - CPU/memory requests and limits.
https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/
[4] kind Documentation, Loading an Image Into Your Cluster.
https://kind.sigs.k8s.io/docs/user/quick-start/#loading-an-image-into-your-cluster
[5] Docker Documentation, Run Containers and Resource Constraints.
https://docs.docker.com/engine/containers/run/ ; https://docs.docker.com/engine/containers/resource_constraints/